Project Title: Comparative Study of Simple RNN, LSTM, and GRU for Amazon Review Sentiment Analysis.

Course: Machine Learning

Course Code: CSE 472

Submitted to:
Raisa Fairooz
Lecturer
Department of Computer Science and Engineering.
Metropolitan University, Sylhet.

Submitted by:
Md Masudul Hasn Akib
ID: 232-115-015

Jibran Masum Didar
ID: 232-115-013

Batch: 59th
Section: A
Department of Computer Science & Engineering.
Metropolitan University, Sylhet.

Date of submission: 15-08-2026

Comparative Study of Simple RNN, LSTM, and GRU for Amazon Review Sentiment Analysis

Md. Masudul Hasan Akib[1], Jibran Masum Didar[2]
Department of Computer Science and Engineering
Metropolitan University, Bangladesh

ABSTRACT

This paper presents an experimental comparison of three recurrent neural network (RNN) architectures — Simple RNN, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) — for binary sentiment classification of Amazon customer reviews. A balanced sample of 50,000 reviews was drawn from the Amazon Reviews Polarity dataset, preprocessed, tokenized, and padded to a fixed sequence length. All three models share an identical downstream architecture (an embedding layer followed by a single recurrent layer, dropout, and a sigmoid output layer) and were trained under matched hyperparameters to ensure a fair comparison. An important finding of this study is the substantial effect of padding masking: initial models trained without mask_zero=True performed close to chance level, while enabling padding masking in the embedding layer produced a large and consistent improvement across architectures. Under the final, masked, and early-stopped configuration, Simple RNN achieved 81.56% test accuracy, LSTM achieved 88.86%, and GRU achieved 90.18%, the highest among the three. The results indicate that gated recurrent architectures, and GRU in particular, achieved the best performance under the experimental conditions used in this study. The paper documents both the exploratory (unmasked) and final (masked) experiments, discusses training and validation behavior, and identifies the limitations and possible extensions of the work.

TABLE OF CONTENTS Page

Contents

LIST OF FIGURES

LIST OF TABLES

ACKNOWLEDGMENT

We would like to express our sincere gratitude to everyone who supported us throughout the completion of this project. This work would not have been possible without the guidance, encouragement, and assistance we received during the course of the project.

First, we would like to thank **Raisa Fairooz, Lecturer, Department of Computer Science and Engineering, Metropolitan University**, for her valuable guidance, constructive feedback, and continuous support throughout the project. Her suggestions helped us improve our understanding of the topic and present our work in a more structured and meaningful way.

We are also grateful to the **Department of Computer Science and Engineering, Metropolitan University**, for providing us with the academic environment and resources necessary to complete this project.

Finally, we would like to thank our friends, classmates, and everyone who supported us directly or indirectly during the development and completion of this project.

KEYWORDS

Sentiment Analysis; Recurrent Neural Networks; LSTM; GRU; Natural Language Processing; Text Classification; Padding Masking; Deep Learning.

1. INTRODUCTION

1.1 Background

Online customer reviews are one of the richest sources of consumer opinion available to businesses and researchers. Automatically determining whether a review expresses a positive or negative sentiment — sentiment analysis — is a well-established task within natural language processing (NLP) and text classification [1]. With the growth of deep learning, recurrent neural networks (RNNs) became a natural fit for sentiment analysis because they process text sequentially and can, in principle, capture dependencies between words across a sentence or review [2]. Variants such as Long Short-Term Memory (LSTM) [3] and Gated Recurrent Unit (GRU) [4] networks were introduced to address the difficulty that simple RNNs have in learning long-range dependencies, using gating mechanisms to control the flow of information through the network.

1.2 Problem Statement

Given the text of an Amazon customer review, the task addressed in this project is binary sentiment classification: predicting whether the review expresses a positive or a negative sentiment. While Simple RNN, LSTM, and GRU are all capable of this task in principle, they differ in architectural complexity and in their ability to retain information over long sequences, which motivates a direct experimental comparison under matched conditions.

1.3 Motivation

Although LSTM and GRU are generally described in the literature as improvements over Simple RNN, the practical size of that improvement depends heavily on the dataset, preprocessing pipeline, and implementation details. In particular, this project encountered a concrete implementation issue — the handling of zero-padding in variable-length sequences — that had a much larger effect on performance than the choice of recurrent architecture itself. Documenting this experimental process, rather than only the final numbers, is a central motivation of this paper.

1.4 Research Question

Among Simple RNN, LSTM, and GRU, which recurrent neural network architecture provides the best performance for Amazon review sentiment classification under the same experimental conditions?

1.5 Objectives

The objectives of this project are: (1) to build a shared preprocessing and modeling pipeline for Amazon review sentiment classification; (2) to implement Simple RNN, LSTM, and GRU models with matched architecture and hyperparameters; (3) to evaluate the effect of padding masking on model performance; and (4) to compare the three architectures on accuracy, precision, recall, F1-score, and parameter count under identical training and evaluation conditions.

1.6 Contributions

This work does not propose a new neural architecture. Its contribution is an undergraduate-level controlled experimental comparison of three well-established recurrent architectures on a shared pipeline, together with an explicit before/after account of how padding masking affected model performance — an implementation detail that is often assumed rather than empirically demonstrated in introductory treatments of RNN-based text classification.

2. RELATED WORK

Sentiment analysis and opinion mining have been studied extensively as a subfield of NLP, with early surveys establishing the core problem formulations, including document-level, sentence-level, and aspect-level sentiment classification [1]. Prior to the widespread adoption of deep learning, sentiment classification relied largely on hand-engineered features combined with traditional classifiers; deep learning approaches later demonstrated that distributed word representations and end-to-end trainable architectures could achieve competitive or superior performance without extensive manual feature engineering [2].

Recurrent neural networks were proposed as a natural architecture for sequential text data because they maintain a hidden state that is updated at every time step. However, simple RNNs are known to suffer from vanishing and exploding gradients, which limits their ability to learn long-range dependencies. The LSTM architecture [3] addressed this limitation through input, forget, and output gates and an explicit memory cell, enabling gradients to propagate over much longer sequences. The GRU architecture [4] later simplified this gating mechanism using only an update gate and a reset gate, reducing the number of parameters relative to LSTM while retaining much of its ability to model long-range dependencies. An empirical comparison of gated recurrent units on sequence modeling tasks found that GRU performed comparably to LSTM despite its simpler structure [5], a finding broadly consistent with the parameter-efficiency observations in the present study.

The dataset used in this project, the Amazon Reviews Polarity dataset, was constructed and popularized as a large-scale text classification benchmark for comparing character-level and word-level models, including recurrent and convolutional architectures [6]. Broader surveys of deep learning-based text classification situate RNN, LSTM, and GRU architectures alongside convolutional and, more recently, transformer-based approaches, noting that recurrent architectures remain a relevant and computationally lighter baseline for sequence classification tasks [2]. More directly relevant to the present study, a recent comparative evaluation of RNN, LSTM, and GRU for sentiment classification of cloud-service reviews reported that gated architectures consistently outperformed the simple RNN baseline, with LSTM and GRU achieving broadly comparable results depending on the dataset and configuration [7].

Research Gap: much of the existing literature compares LSTM and GRU on tasks such as machine translation, speech, or music modeling rather than on large-scale product review sentiment classification with a controlled, from-scratch embedding setup. In addition, practical implementation factors — such as the handling of padded sequences — are rarely reported quantitatively, even though, as shown in this study, they can dominate the performance difference between architectures. This project addresses that gap at an undergraduate, course-project scale by directly reporting both the unmasked and masked experimental configurations.

3. METHODOLOGY

3.1 Overall Workflow

The overall workflow consists of the following stages, applied identically to all three models: dataset loading and sampling; text preprocessing; label encoding; stratified train/validation/test splitting; tokenization; sequence padding; word embedding (with and without padding masking, in the exploratory

and final configurations respectively); recurrent model training; and evaluation using accuracy, precision, recall, F1-score, and confusion matrices.

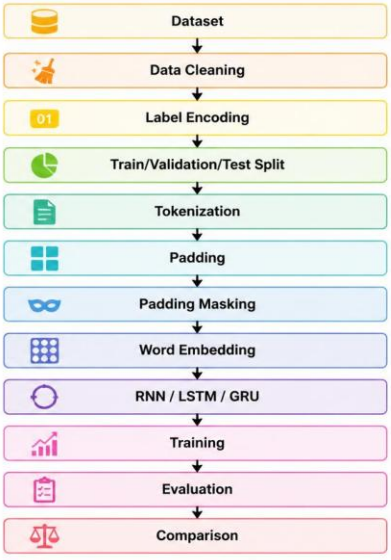

Figure 3.1. Methodology workflow.

3.2 Dataset Description

The dataset used is the Amazon Reviews Polarity dataset, sourced from the Kaggle repository bittlingmayer/amazonreviews [6]. The full training file contains 3,600,000 reviews, perfectly balanced between the two classes (__label__1 = negative, __label__2 = positive). Due to computational constraints typical of a course project, a random sample of 50,000 reviews was drawn (fixed random seed 42) for use in this study. The sampled subset is close to balanced, containing 25,039 positive and 24,961 negative reviews. No missing values or duplicate rows were present in the sampled data.

Table 3.1. Dataset Statistics

Statistic	Value
Full dataset size	3,600,000 reviews
Full dataset balance	1,800,000 / 1,800,000
Sampled dataset size	50,000 reviews
Sampled class balance	25,039 positive / 24,961 negative
Mean review length (words)	78.6
Review length range (words)	13 – 210

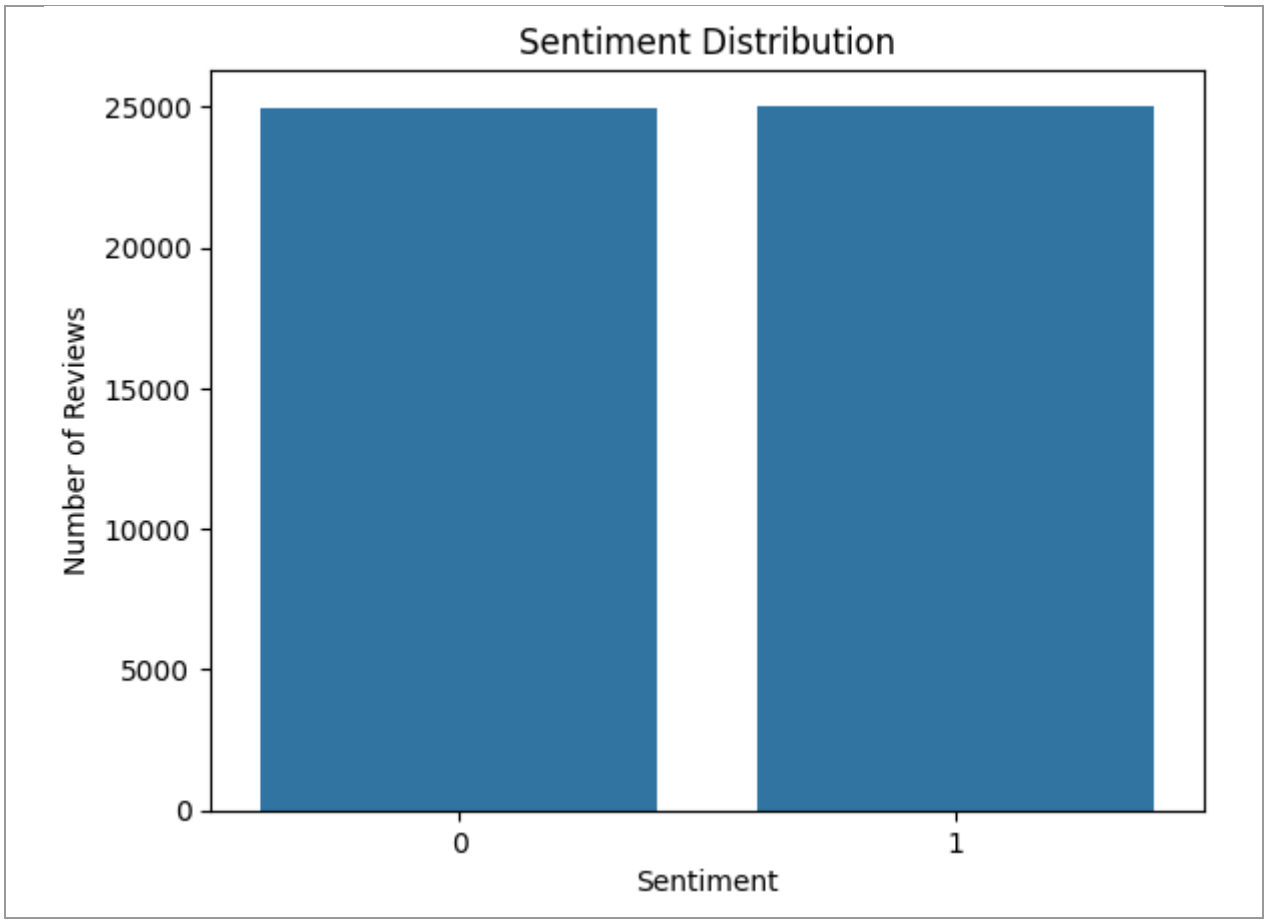

Figure 3.2. Distribution of positive and negative reviews in the sampled dataset.

3.3 Data Preprocessing

Text preprocessing was applied in the following order: (1) lowercasing of all review text; (2) removal of HTML tags using a regular expression; (3) removal of all non-alphabetic characters, replacing them with whitespace; (4) collapsing repeated whitespace into single spaces and trimming leading/trailing whitespace. Rows that became empty strings after cleaning were removed from the dataset, although no such rows were found in the sampled data.

3.4 Label Encoding

The raw dataset labels, __label__1 and __label__2, were mapped to binary integer labels: __label__1 was mapped to 0 (Negative) and __label__2 was mapped to 1 (Positive).

3.5 Dataset Splitting

The 50,000-review sample was split into training, validation, and test sets using stratified sampling to preserve class balance, with a fixed random state of 42. The data was first split 80/20 into a training set and a temporary set; the temporary set was then split evenly (50/50) into validation and test sets.

Table 3.2. Dataset Split

Split	Number of Samples	Proportion
Training	40,000	80%
Validation	5,000	10%
Test	5,000	10%

3.6 Tokenization

Text was tokenized using the Keras TextVectorization layer, configured with a maximum vocabulary size of 20,000 tokens and integer output mode. The vectorizer was fit (adapted) exclusively on the training set to avoid information leakage from the validation and test sets.

3.7 Padding

All tokenized sequences were padded or truncated to a fixed maximum length of 200 tokens using the output_sequence_length parameter of TextVectorization. This produced fixed-size input tensors of shape (batch_size, 200) for all three data splits.

3.8 Padding Masking

Because reviews vary substantially in length, most sequences contain trailing zero-padding tokens after truncation/padding to length 200. In the exploratory phase of this project, the embedding layer did not mask these padding tokens, meaning the recurrent layers processed padding positions as if they were meaningful input. In the final configuration, the embedding layer was instantiated with mask_zero=True, which propagates a mask through the recurrent layer so that padding positions are excluded from the recurrent computation. As detailed in Section 5.2, this single change was associated with a very large improvement in model performance for both Simple RNN and LSTM.

3.9 Word Embedding

All three models use a Keras Embedding layer with an input dimension equal to the vocabulary size (20,000) and an output (embedding) dimension of 128. Embeddings were learned from scratch during training; no pretrained word vectors (e.g., Word2Vec or GloVe) were used.

3.10 Simple RNN

The Simple RNN model consists of an Embedding layer, a SimpleRNN layer with 64 units, a Dropout layer with rate 0.3, and a Dense output layer with a single sigmoid-activated unit. The final (masked) Simple RNN model has 2,572,417 trainable parameters.

3.11 LSTM

The LSTM model follows the same structure, replacing the SimpleRNN layer with an LSTM layer of 64 units. The final (masked) LSTM model has 2,609,473 trainable parameters.

3.12 GRU

The GRU model follows the same structure, replacing the recurrent layer with a GRU layer of 64 units. The GRU model has 2,597,313 trainable parameters, positioning it between Simple RNN and LSTM in terms of model size.

3.13 Training Configuration

All models were compiled with the Adam optimizer and binary cross-entropy loss, and trained with a batch size of 64. The final models were trained for up to 10 epochs using EarlyStopping (monitor: validation loss; patience: 2 epochs; restore_best_weights: True) to prevent overfitting and to automatically select the best-performing epoch's weights.

3.14 Evaluation Metrics

Model performance was evaluated on the held-out test set using accuracy, precision, recall, and F1-score, computed with scikit-learn [8], and visualized using confusion matrices. For a binary classification problem with true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), the metrics are defined as:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}Score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

4. EXPERIMENTAL SETUP

4.1 Development Environment

All experiments were conducted in Google Colab using a Python runtime.

4.2 Software and Libraries

The implementation used TensorFlow/Keras for model definition and training, pandas and NumPy for data handling, scikit-learn for train/test splitting and evaluation metrics [8], and Matplotlib/Seaborn for visualization.

4.3 Dataset Split

As described in Section 3.5, the dataset was split into 40,000 training, 5,000 validation, and 5,000 test samples using stratified sampling with a fixed random state.

4.4 Hyperparameters

Table 4.1. Common Hyperparameters and Model Configuration

Hyperparameter	Value
Vocabulary size	20,000
Maximum sequence length	200
Embedding dimension	128

Recurrent units	64
Dropout rate	0.3
Optimizer	Adam
Loss function	Binary cross-entropy
Batch size	64
Maximum epochs	10
Early stopping	Yes (patience = 2, monitor = val_loss)
Padding masking (final models)	Yes (mask_zero = True)

4.5 Model Configuration

All three models share an identical downstream architecture (Embedding → recurrent layer → Dropout → Dense), differing only in the type of recurrent layer used. This design isolates the recurrent unit as the primary variable under comparison.

4.6 Training Procedure

Each final model was trained on the same training set and validated on the same validation set, with EarlyStopping restoring the weights from the epoch with the lowest validation loss. Test-set evaluation was performed only after training was complete, using model.evaluate() followed by explicit prediction and thresholding (probability ≥ 0.5 assigned to the positive class) for computing scikit-learn metrics.

4.7 Reproducibility and Random Seed

A fixed random seed (SEED = 42) was set for Python's random module, NumPy, and TensorFlow (random.seed(SEED), np.random.seed(SEED), tf.random.set_seed(SEED)) prior to data sampling, splitting, and model training. Under this fixed-seed configuration, a full re-execution of the notebook reproduced identical test-set results for all three final models, which is reported here as an empirical observation of stability in this particular setup. This should not be read as a general claim that all TensorFlow or GPU operations are guaranteed to be bit-for-bit deterministic across all hardware and software configurations; it reflects the reproducibility observed specifically in the environment used for this project.

4.8 Fairness of Model Comparison

The three models were kept identical in every respect other than the recurrent layer type: the same sampled dataset, the same train/validation/test split, the same preprocessing and tokenization pipeline, the same embedding configuration and padding masking, the same optimizer, loss function, batch size, and early-stopping strategy. This design supports a like-for-like comparison of the recurrent units themselves, though it does not constitute an exhaustive hyperparameter search for any individual architecture (see Section 7).

5. RESULTS AND DISCUSSION

5.1 Initial Experimental Results

Before padding masking was introduced, exploratory versions of the Simple RNN and LSTM models were trained without mask_zero=True. The Simple RNN model, trained for 5 epochs without early stopping, achieved a test accuracy of 65.58% (model.evaluate() output). The unmasked LSTM model, also trained for 5 epochs, achieved a test accuracy of 50.14%; its classification report showed a recall of approximately 0.00 for the negative class, indicating that the model had effectively collapsed to predicting the positive class for nearly all test samples. These results indicated that both unmasked models were failing to learn a meaningful distinction between sentiment classes, motivating further investigation.

Table 5.1. Initial (Unmasked) Exploratory Results

Model (unmasked, exploratory)	Test Accuracy
Simple RNN	65.58%
LSTM	50.14%

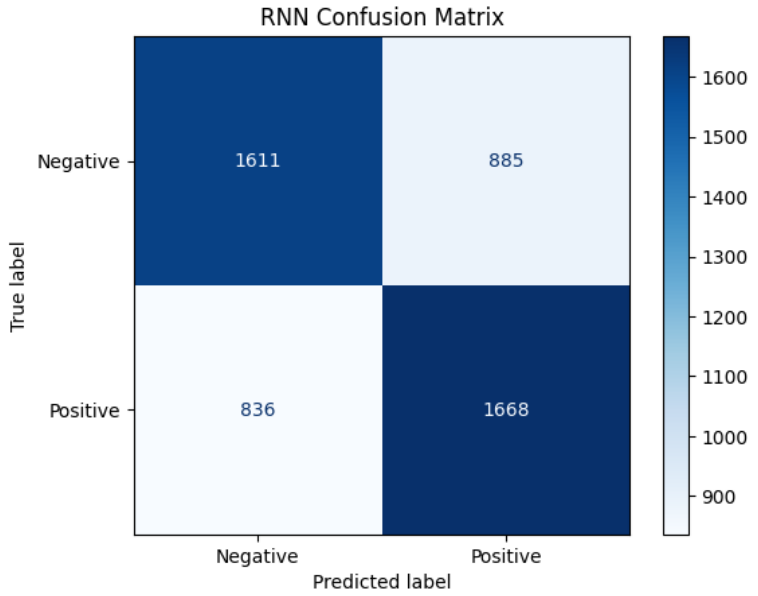

Figure 5.1. Initial (unmasked) Simple RNN confusion matrix on the test set.

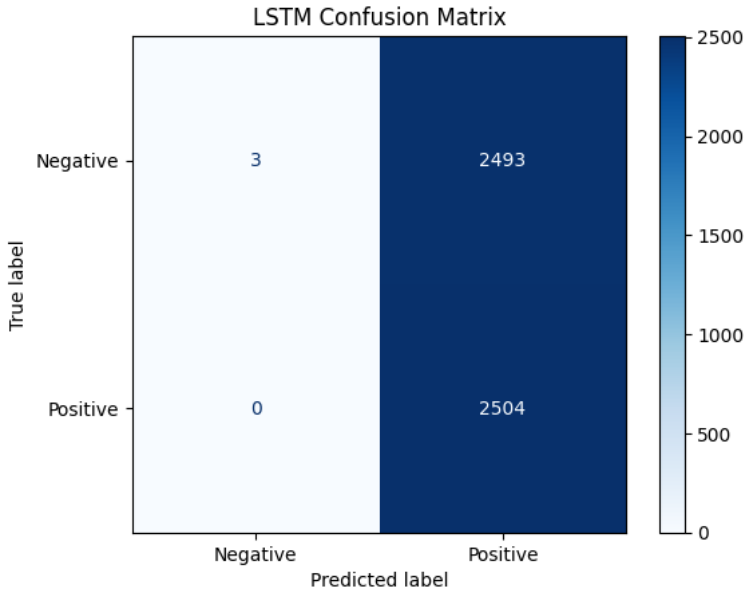

Figure 5.2. Initial (unmasked) LSTM confusion matrix on the test set.

5.2 Effect of Padding Masking

Investigation of the unmasked models' behavior indicated that the recurrent layers were processing the trailing zero-padding tokens present in most sequences (since most reviews are shorter than the fixed sequence length of 200 tokens) as if they were meaningful input. Introducing mask_zero=True in the Embedding layer allows Keras to propagate a mask through the recurrent layer, causing padding positions to be excluded from the recurrent computation. After this change, both the Simple RNN and LSTM models showed a substantial and consistent improvement in training and validation accuracy from the first training epoch onward, in contrast to the near-chance-level performance observed without masking. This is treated in this paper as the primary observed effect of padding masking; a plausible explanation is that, without masking, the padding positions diluted or corrupted the final hidden state used for classification, particularly for shorter reviews with proportionally more padding — this explanation is consistent with, but not independently verified beyond, the observed accuracy improvement.

5.3 Final Model Performance

The final models — using padding masking and EarlyStopping — were evaluated on the same held-out test set of 5,000 reviews. The verified results, obtained directly from model.evaluate() and cross-checked against scikit-learn's accuracy_score(), precision_score(), recall_score(), and f1_score() on the same predictions, are shown in Table 5.2.

Table 5.2. Final Performance Comparison (Masked, Early-Stopped Models)

Model	Accuracy	Precision	Recall	F1-Score	Parameters
Simple RNN	81.56%	83.38%	78.91%	81.08%	2,572,417
LSTM	88.86%	88.92%	88.82%	88.87%	2,609,473
GRU	90.18%	91.68%	88.42%	90.02%	2,597,313

For each model, the accuracy reported by model.evaluate() matched the accuracy computed independently by scikit-learn's accuracy_score() on the same test predictions (81.56% for Simple RNN, 88.86% for LSTM, 90.18% for GRU), confirming internal consistency between the two evaluation paths. Under this

experimental configuration, GRU achieved the highest accuracy, precision-weighted F1-score, and overall balance between precision and recall, followed by LSTM and then Simple RNN.

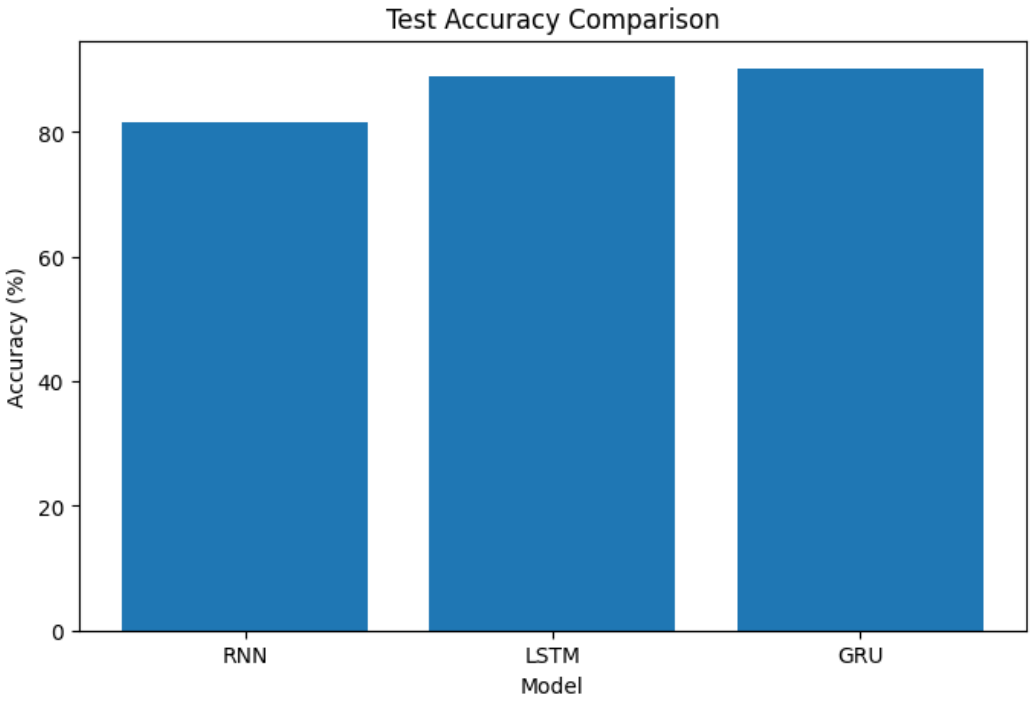

Figure 5.3. Test accuracy comparison across Simple RNN, LSTM, and GRU.

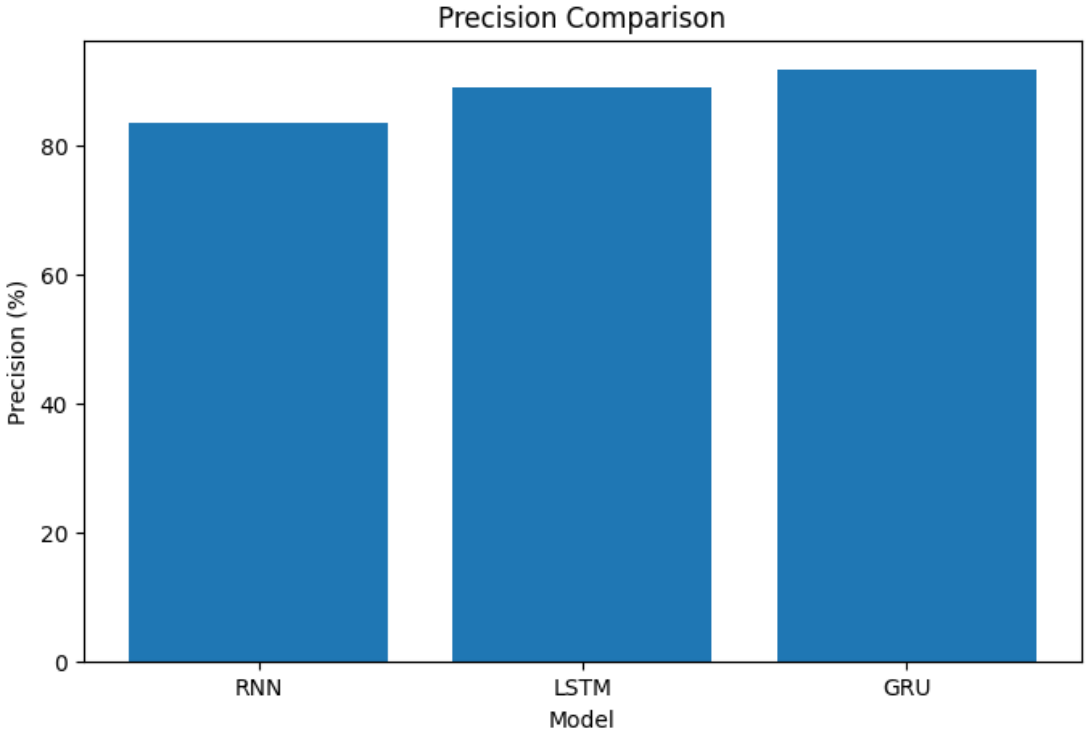

Figure 5.4. Precision comparison across Simple RNN, LSTM, and GRU.

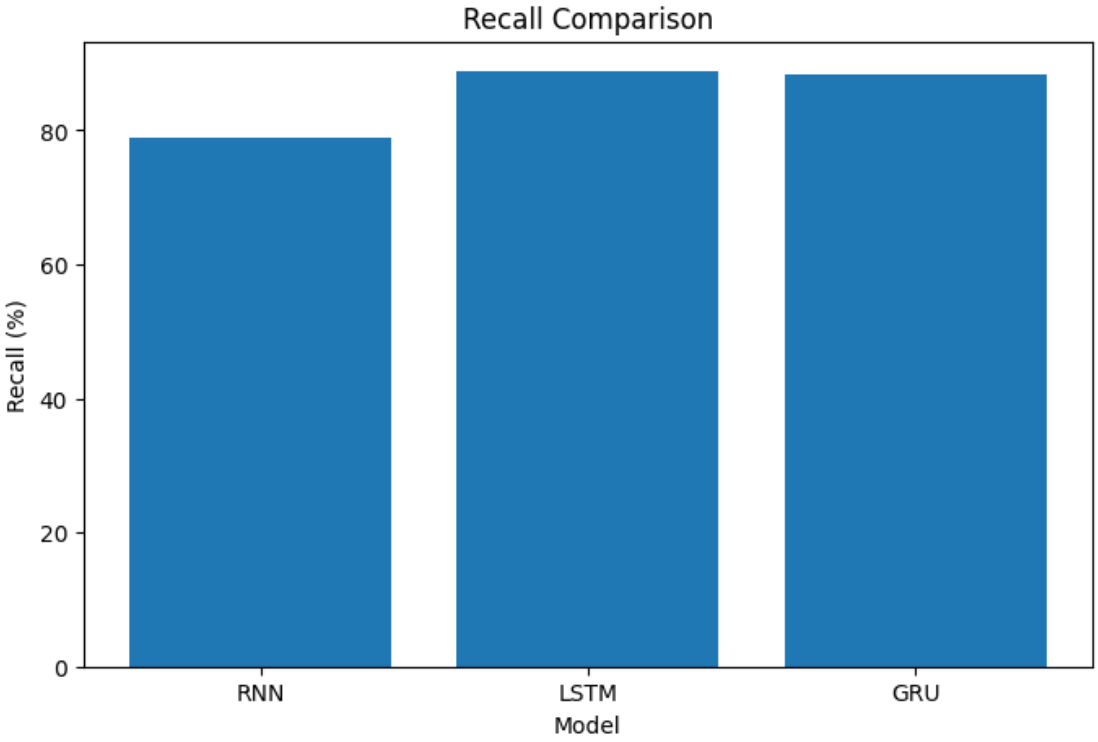

Figure 5.5. Recall comparison across Simple RNN, LSTM, and GRU.

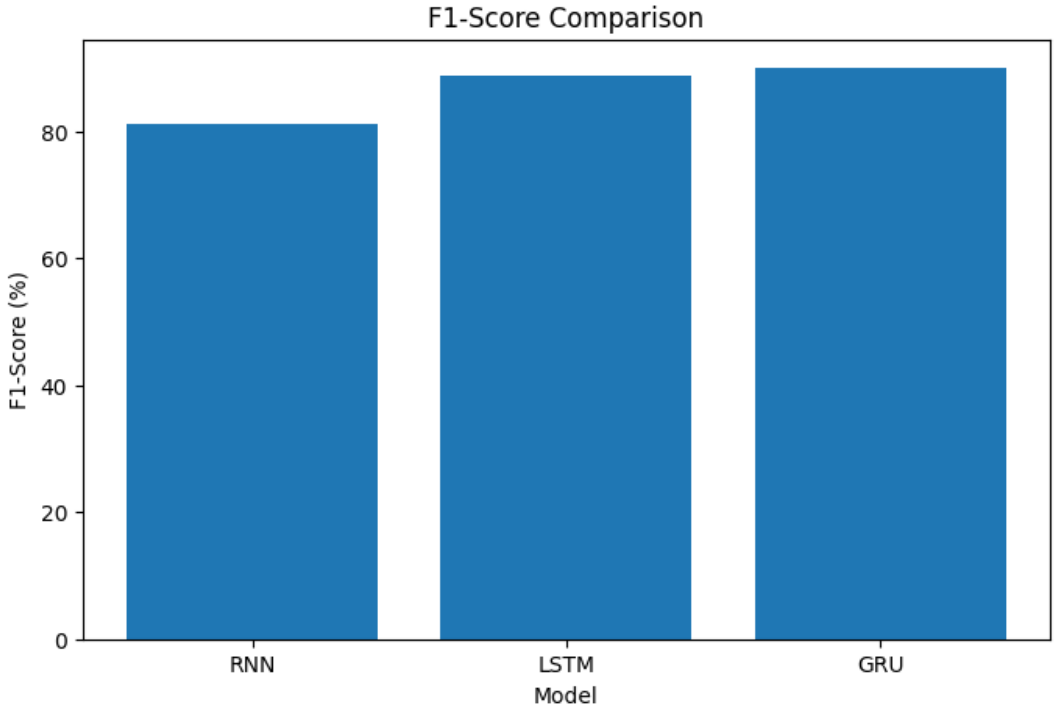

Figure 5.6. F1-score comparison across Simple RNN, LSTM, and GRU.

5.4 Confusion Matrix Analysis

The confusion matrices for the three final (masked, early-stopped) models, computed on the 5,000-sample test set, are shown below.

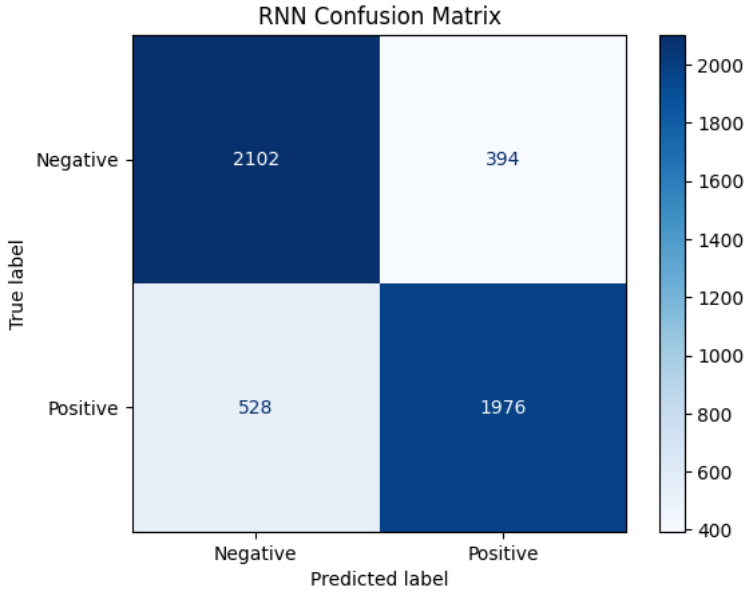

Figure 5.7. Final (masked) Simple RNN confusion matrix on the test set.

This confusion matrix is fully consistent with the metrics reported in Table 5.2: with 2,102 true negatives, 394 false positives, 528 false negatives, and 1,976 true positives, accuracy = (2,102 + 1,976) / 5,000 = 81.56%, precision = 1,976 / (1,976 + 394) = 83.38%, and recall = 1,976 / (1,976 + 528) = 78.91% — each matching Table 5.2 exactly.

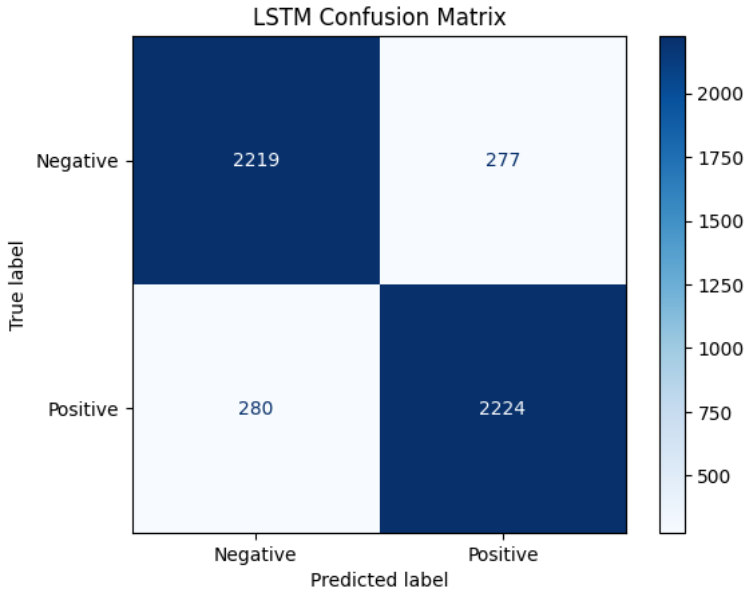

Figure 5.8. Final (masked) LSTM confusion matrix on the test set.

With 2,219 true negatives, 277 false positives, 280 false negatives, and 2,224 true positives, accuracy = (2,219 + 2,224) / 5,000 = 88.86%, precision = 2,224 / (2,224 + 277) = 88.92%, and recall = 2,224 / (2,224 + 280) = 88.82% — again matching Table 5.2 exactly.

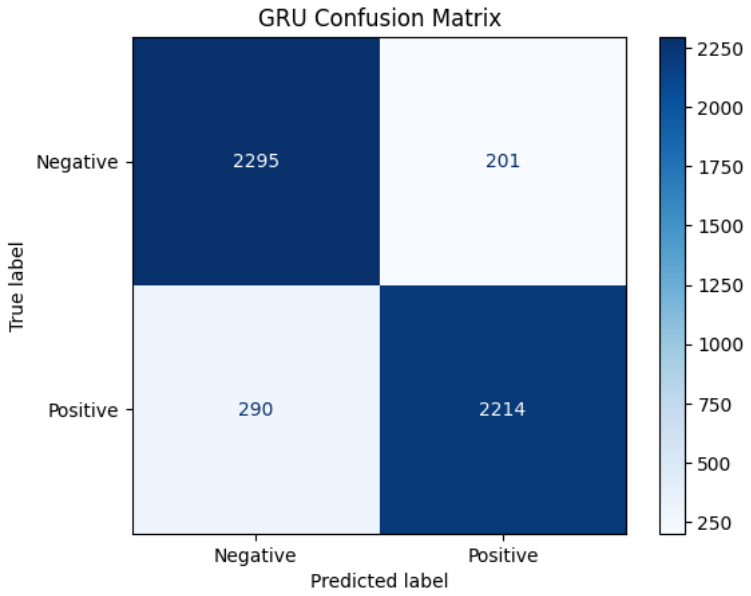

Figure 5.9. Final (masked) GRU confusion matrix on the test set.

With 2,295 true negatives, 201 false positives, 290 false negatives, and 2,214 true positives, accuracy = (2,295 + 2,214) / 5,000 = 90.18%, precision = 2,214 / (2,214 + 201) = 91.68%, and recall = 2,214 / (2,214 + 290) = 88.42% — matching Table 5.2 exactly. The GRU model produced a moderately higher number of false negatives (290) than false positives (201), indicating a slight tendency to under-predict the positive class relative to the negative class on this test set.

Across all three final models, the confusion matrices are fully reconciled against the accuracy, precision, recall, and F1-score values reported in Table 5.2, confirming that the reported metrics, model.evaluate() outputs, and confusion matrices all derive from the same underlying predictions on the same test set.

5.5 Training and Validation Performance

The training and validation behavior was examined at different stages of the experiments. For the **initial unmasked RNN**, training accuracy gradually increased, while validation accuracy fluctuated considerably across epochs. The validation loss also showed noticeable variation, indicating unstable learning during the initial experiment.

For the **initial unmasked LSTM**, both training and validation accuracy remained close to **50%**, while the loss values changed only slightly. This behavior was consistent with the near-random performance observed in the initial LSTM evaluation.

For the **final masked GRU**, training accuracy increased steadily, while validation accuracy reached its highest point around **epoch 2** and then slightly decreased. Similarly, validation loss reached its lowest point around epoch 2 before increasing in later epochs, indicating some overfitting.

The final masked **RNN and LSTM** training metrics were recorded in the notebook, but corresponding curves were not plotted. Therefore, their final training behavior is discussed using the recorded metrics

rather than these figures. **EarlyStopping** was used in the final models to monitor validation loss and restore the best-performing weights.

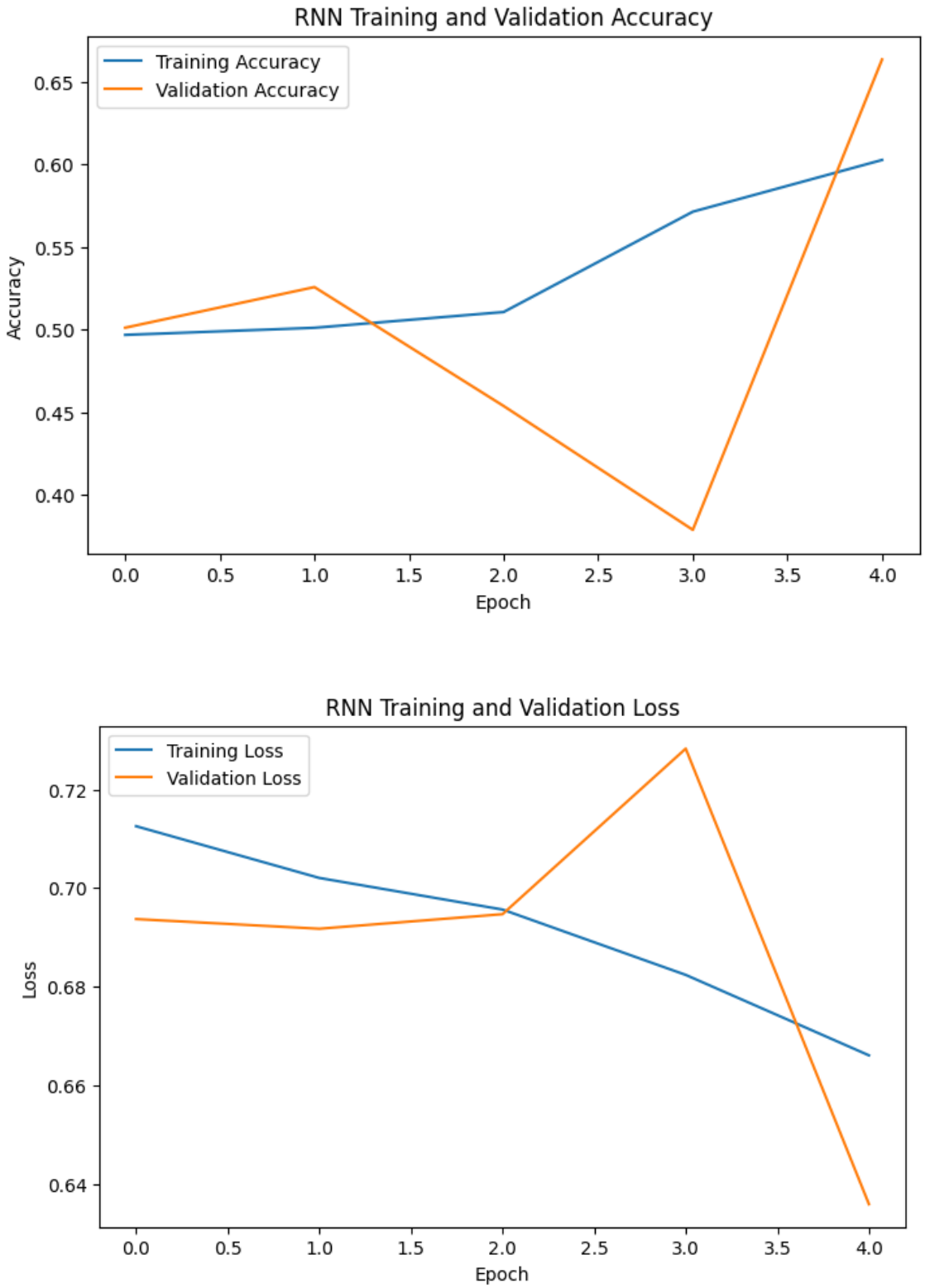

Figure 5.10. Simple RNN training and validation accuracy and loss curves.

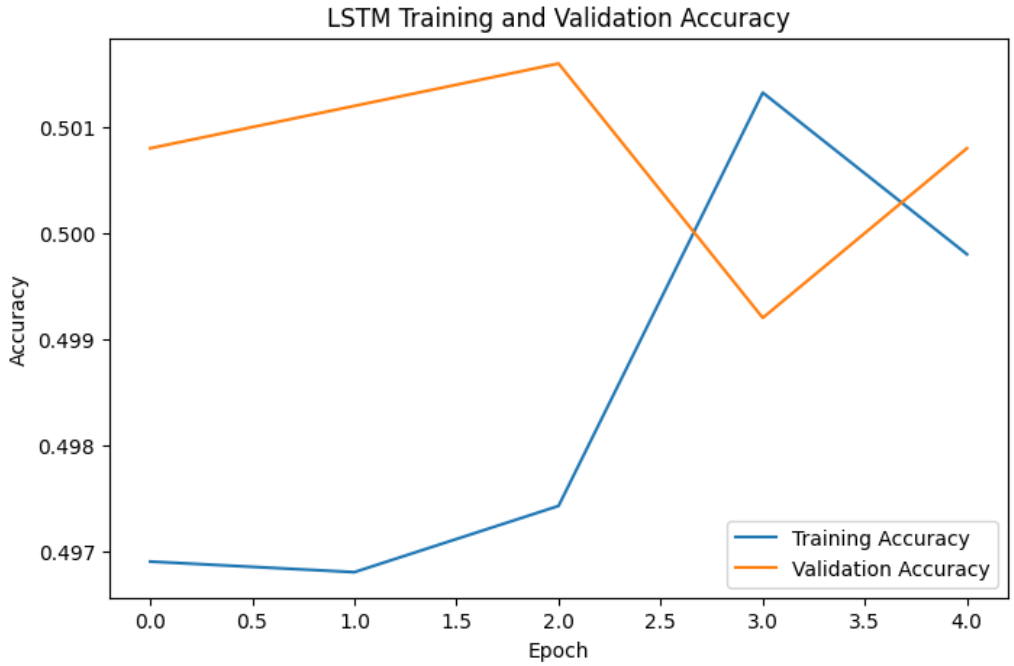

Figure 5.11. LSTM training and validation accuracy.

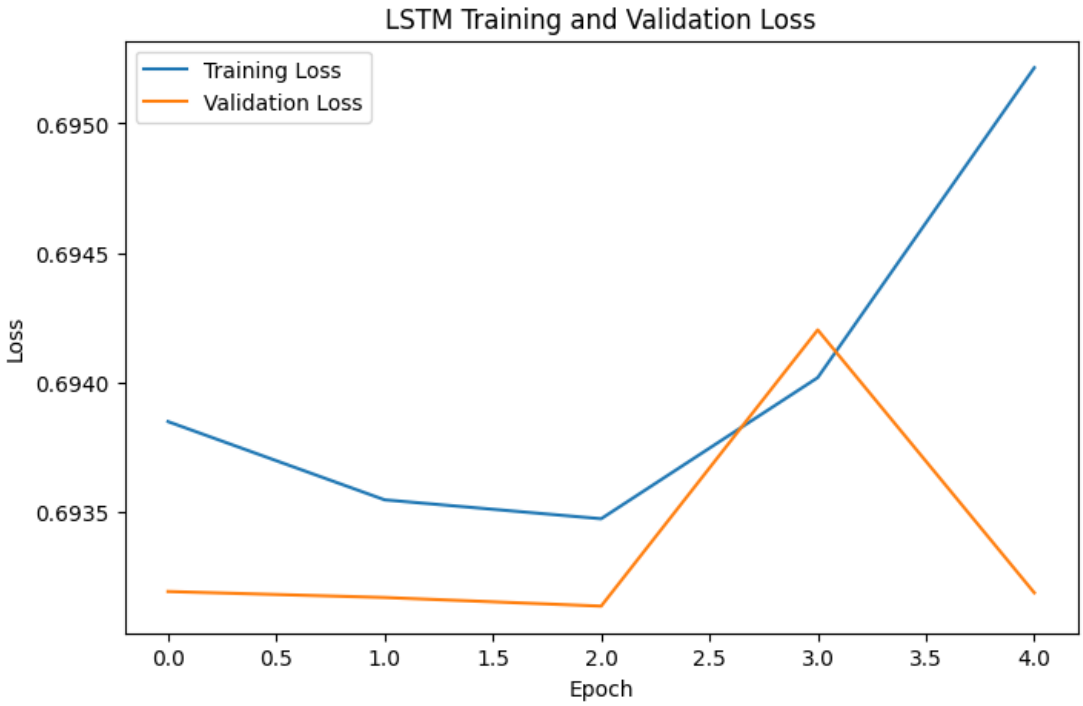

Figure 5.12. LSTM training and validation loss curve.

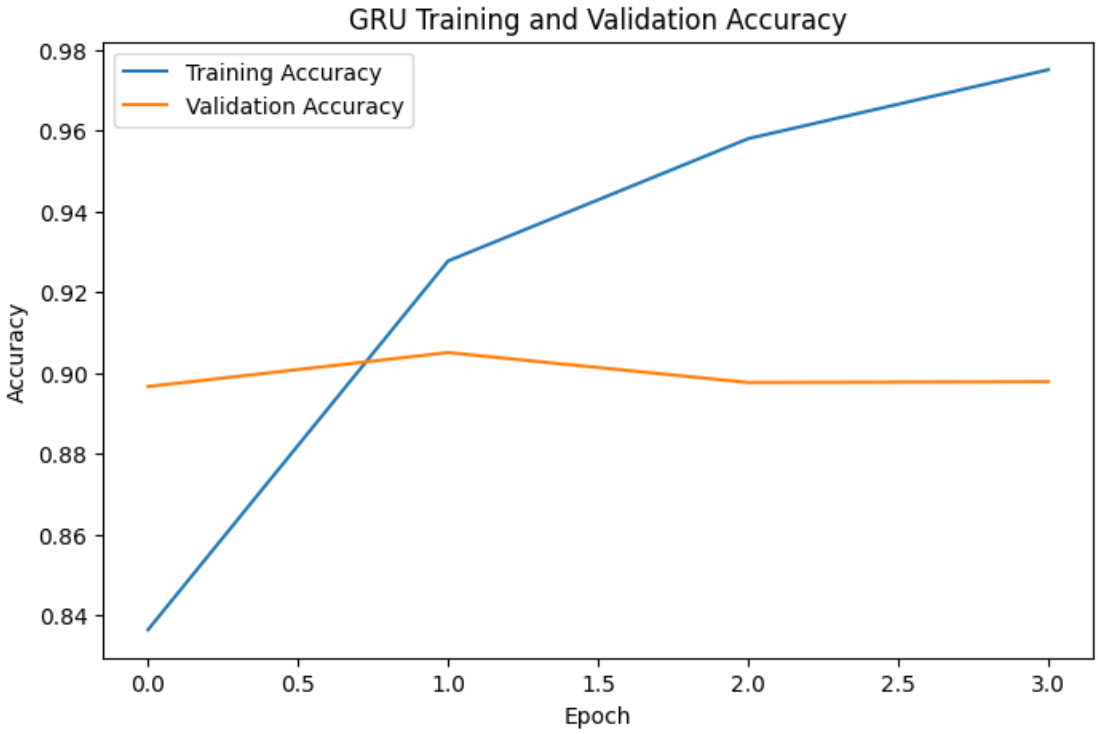

Figure 5.13. GRU training and validation accuracy curves.

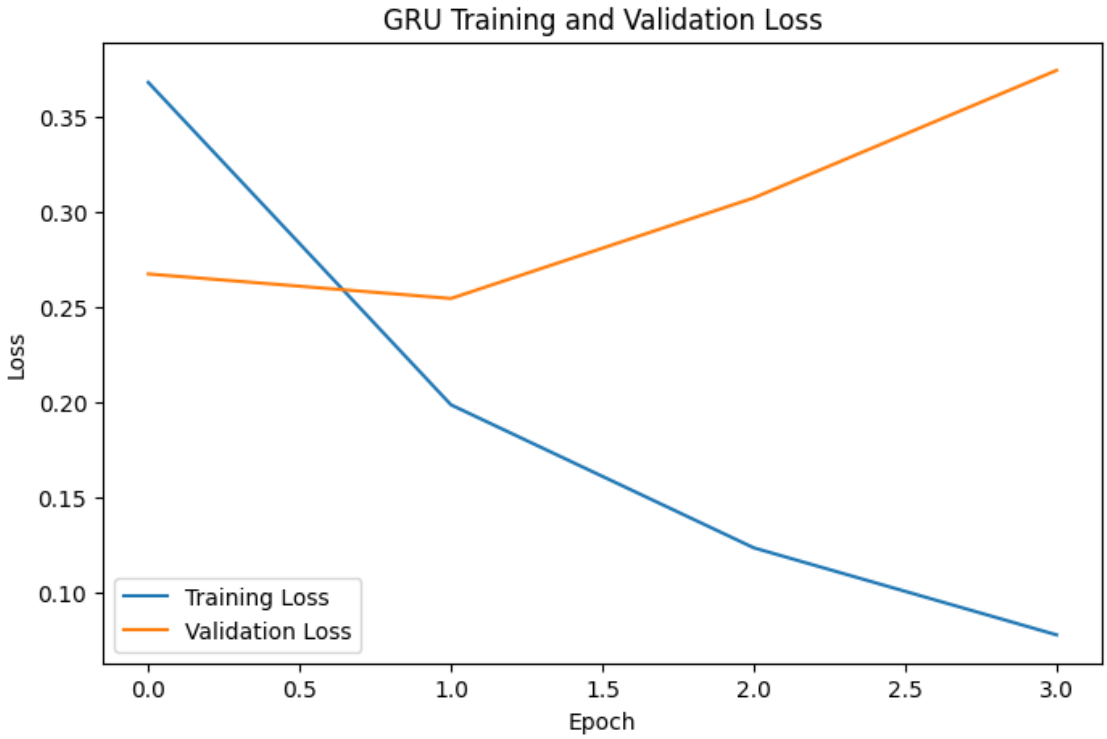

Figure 5.14. GRU training and validation loss curves.

5.6 Parameter Comparison

The three models are close in overall size, since the embedding layer (2,560,000 parameters, common to all three) dominates the total parameter count. The recurrent layer itself is where the models differ: Simple RNN contributes the fewest additional parameters (12,352), GRU contributes an intermediate number (37,248), and LSTM contributes the most (49,408), consistent with LSTM's more complex three-gate structure compared to GRU's two-gate structure and Simple RNN's single recurrent transformation. Despite having fewer recurrent-layer parameters than LSTM, GRU achieved the highest accuracy in this study, suggesting that, for this task and dataset, GRU's gating mechanism was at least as effective as LSTM's more complex gating while requiring somewhat less model capacity.

5.7 Comparative Discussion

Observed result: under the experimental configuration used in this study, GRU achieved the highest test accuracy (90.18%), followed by LSTM (88.86%) and Simple RNN (81.56%).

Possible explanation: gated architectures (LSTM and GRU) are designed to mitigate the vanishing-gradient limitations of Simple RNN, which may explain their consistently higher performance relative to Simple RNN in this experiment. The relatively small gap between LSTM and GRU is broadly consistent with prior empirical comparisons reporting that GRU performs comparably to LSTM on various sequence modeling tasks despite its simpler gating structure [5], and with a recent comparative study of RNN, LSTM, and GRU for review sentiment classification that likewise found gated architectures to consistently outperform the simple RNN baseline [7]. However, this explanation is offered as a plausible interpretation of the observed results, not as an independently tested causal mechanism within this study.

It is important to state clearly that these results should not be interpreted as evidence that GRU is universally superior to LSTM. GRU achieved the best performance under the specific experimental configuration used in this study — a fixed vocabulary size, sequence length, embedding dimension, unit count, dataset, and training procedure — and a different configuration or dataset could plausibly yield a different ranking between LSTM and GRU.

5.8 Custom Review Predictions

As a final qualitative check, the three trained models were used to predict the sentiment of five unseen, manually written reviews. All three models correctly classified all five reviews as positive or negative, with sigmoid probabilities generally close to 0 or 1, indicating confident predictions on these clear-cut examples. A compact summary is shown below; the complete results, including all prediction probabilities, are provided in Appendix A.

Table 5.3. Custom Review Prediction Summary

Review (abbreviated)	RNN	LSTM	GRU
“This product is amazing...”	Positive	Positive	Positive
“Terrible product. I regret...”	Negative	Negative	Negative
“The product is okay, but...”	Negative	Negative	Negative
“Excellent quality and fast...”	Positive	Positive	Positive
“Waste of money. Very...”	Negative	Negative	Negative

It is important to note that this exercise demonstrates practical inference behavior only; it is not a substitute for quantitative evaluation, since accuracy as a metric requires a sufficiently large set of samples with

known ground-truth labels, which five hand-picked examples without independently verified ground-truth labels do not provide.

6. CONCLUSION

This project implemented and compared Simple RNN, LSTM, and GRU architectures for binary sentiment classification of Amazon customer reviews, using a shared preprocessing pipeline, matched hyperparameters, and identical training and evaluation procedures. An important experimental finding was that padding masking (mask_zero=True) had a large effect on model performance: unmasked models performed close to chance level, while the corresponding masked models achieved substantially higher accuracy. Under the final, masked, early-stopped configuration, Simple RNN achieved 81.56% test accuracy, LSTM achieved 88.86%, and GRU achieved 90.18%, with GRU also achieving the highest precision, recall, and F1-score among the three models while using fewer recurrent-layer parameters than LSTM. These results indicate that, under the experimental configuration used in this study, GRU provided the best sentiment classification performance among the three recurrent architectures we evaluated.

7. LIMITATIONS AND FUTURE WORK

Limitations

This study has several limitations. First, a fixed vocabulary size (20,000) and maximum sequence length (200) were used throughout; different values could affect results and were not explored. Second, only a single architecture configuration (64 recurrent units, one recurrent layer) was evaluated per model type, without broader hyperparameter tuning. Third, embeddings were learned from scratch rather than initialized from pretrained word vectors. Fourth, all experiments were conducted on a single 50,000-review sample of one dataset, so results may not generalize to the full 3.6-million-review dataset or to other review domains. Finally, computational constraints limited the scale and number of training runs that could be performed within the scope of this course project.

Future Work

Future extensions of this work could include systematic hyperparameter optimization for each architecture; evaluation of bidirectional RNN, LSTM, and GRU variants; incorporation of pretrained word embeddings such as Word2Vec or GloVe; the addition of attention mechanisms; comparison against transformer-based models such as BERT; and evaluation on the full-scale dataset or on additional sentiment analysis datasets to assess generalization.

APPENDIX A — COMPLETE CUSTOM REVIEW PREDICTION RESULTS

The table below lists the complete results of testing the three trained models (Simple RNN, LSTM, GRU) on five unseen, manually written reviews. Predicted sentiment and the model's sigmoid output probability (probability of the positive class) are reported for each model. These reviews do not have independently verified ground-truth labels; they are provided as a qualitative demonstration of inference behavior only and are not used as an accuracy measurement anywhere in this paper.

Table A.1. Complete Custom Review Prediction Results

Review	RNN (Pred, Prob.)	LSTM (Pred, Prob.)	GRU (Pred, Prob.)
This product is amazing. I really loved it!	Positive, 0.9865	Positive, 0.9585	Positive, 0.9960
Terrible product. I regret buying it.	Negative, 0.0802	Negative, 0.0794	Negative, 0.0355
The product is okay, but nothing special.	Negative, 0.0935	Negative, 0.1355	Negative, 0.0966
Excellent quality and fast delivery.	Positive, 0.9840	Positive, 0.9798	Positive, 0.9952
Waste of money. Very disappointing.	Negative, 0.0016	Negative, 0.0003	Negative, 0.0004

REFERENCES

[1] B. Pang and L. Lee, “Opinion Mining and Sentiment Analysis,” Foundations and Trends® in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.

[2] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, and J. Gao, “Deep Learning Based Text Classification: A Comprehensive Review,” arXiv preprint arXiv:2004.03705, 2020.

[3] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[4] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in Proc. 2014 Conf. Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 2014, pp. 1724–1734.

[5] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling,” arXiv preprint arXiv:1412.3555, 2014.

[6] X. Zhang, J. Zhao, and Y. LeCun, “Character-level Convolutional Networks for Text Classification,” in Advances in Neural Information Processing Systems 28 (NIPS 2015), 2015.

[7] M. R. Raza, W. Hussain, and J. M. Merigó, “Cloud Sentiment Accuracy Comparison using RNN, LSTM and GRU,” in 2021 Innovations in Intelligent Systems and Applications Conference (ASYU), Elazığ, Turkey, 2021, pp. 1–5, doi: 10.1109/ASYU52992.2021.9599044.

[8] scikit-learn developers, “sklearn.metrics: Classification metrics,” scikit-learn Documentation. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html

[9] TensorFlow, “tf.keras.layers.Embedding,” TensorFlow API Documentation. [Online]. Available: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Embedding